Documento de Arquitectura de Software

19 de octubre de 2026

2026

Documento de Arquitectura de Software (DAS) de la API REST Patent Sweep, servicio de vigilancia de patentes de la plataforma de automatización de la cadena de suministro orientada a servicios de Cutit Saws Ltd. Curso 750021C Desarrollo de Software II, Universidad del Valle, 2026-2.

Cutit Saws Ltd.

Control de Cambios

Creación del documento				
Autor	Isabela Bermúdez Moreno, Andrés Gerardo González Rosero, Juan Marín Orozco, David Taborda Montenegro y Victoria Yuan Chen		**Fecha**	13/10/2026
Revisado por	**Nombre**	David Taborda Montenegro	**Fecha**	14/10/2026
	Cargo	Scrum Máster		
Aprobado por	**Nombre**	David Taborda Montenegro	**Fecha**	19/10/2026
	Cargo	Scrum Máster		

Versión	Descripción	Autor	Aprobado Por	Fecha Aprobación
1.0	Versión inicial: secciones 1, 2.2, 2.3, 3, 4 y 5 a partir del PB Patent Sweep V1.0.	Isabela Bermúdez Moreno y Juan Marín Orozco	Andrés Gerardo González Rosero	13/10/2026
2.0	Definiciones ISO/IEC 25010:2023 en las tablas de 2.1, asignación de patrones de diseño y elaboración de diagramas.	Victoria Yuan Chen y Andrés Gerardo González Rosero	Isabela Bermúdez Moreno y Juan Marín Orozco	14/10/2026
3.0	Corrección de datos consignados, mejora de diagramas y rellenado de secciones pendientes.	Victoria Yuan Chen, Juan Marín Orozco y David Taborda Montenegro	David Taborda Montenegro	19/10/2026

TABLA DE CONTENIDO

1. INTRODUCCIÓN 4

1.1 Propósito 4
1.2 Alcance 4
1.2.1. Módulos y Funcionalidades 4
1.2.2. Decisiones arquitecturales 5
1.2.3. Supuestos y pendientes de integración 7
1.2.4. Alcance del documento 8
1.3 Definiciones, siglas y abreviaturas 9
1.4 Referencias 10
1.5 Vista Global 11

2. MACRO ARQUITECTURA 11

2.1 Metas y Restricciones Arquitectónicas 14
2.2 Patrones de diseño 39
2.3 Atributos de calidad vs. patrones 42

3. VISTA FÍSICA 43

4. VISTA FUNCIONAL O LÓGICA 48

4.1 Modos de comunicación 48
4.2 Redes de procesos 48
4.5 Interacciones 49
4.6 Contrato REST 49

5. VISTA DE DESPLIEGUE 51

5.1 Configuración de producción en AWS 51
5.2 Mapa de procesos por nodo 52

1. Introducción

Este documento describe la arquitectura de software de la **API REST Patent Sweep**, uno de los cinco servicios que conforman la plataforma backend orientada a servicios (SOA) de Cutit Saws Ltd. La API vigila la propiedad intelectual relacionada con las fórmulas de cuchilla: ejecuta barridos periódicos de patentes sobre las fórmulas activas, permite búsquedas y evaluaciones puntuales, registra los conflictos detectados y expone el estado de cada fórmula para que Fabricación decida si es seguro producirlo.

El DAS presenta el propósito y el alcance del documento, las decisiones arquitecturales, los atributos de calidad que guían el diseño (sección 2.1), los patrones que los materializan y la representación de la solución mediante las vistas del modelo 4+1 de Kruchten.

1.1 Propósito

El DAS es la referencia técnica del equipo de Patent Sweep: documenta las decisiones que condicionan la construcción, el despliegue, la operación y la evolución de la API, y las justifica a partir del Product Backlog (PB) V1.0 y del caso de estudio. Su audiencia y uso esperado son:

- **Integrantes del equipo:** guía de implementación (capas, componentes, contratos REST y tácticas de calidad).
- **Equipos de Materiales/Fórmulas, Fabricación y Notificaciones:** contrato de integración (endpoints, códigos de respuesta, callback de dictamen y supuestos pendientes de acordar, tabla 3).
- **Profesor y revisores:** evidencia de trazabilidad entre historias de usuario, atributos de calidad, patrones y vistas.

La sección 1 introduce el documento; la sección 2 presenta la macro arquitectura, los atributos de calidad (2.1), los patrones de diseño (2.2) y su relación (2.3); las secciones 3, 4 y 5 describen las vistas física, lógica y de despliegue.

1.2 Alcance

El DAS cubre exclusivamente el backend de la API Patent Sweep y su proceso de ingeniería: versionamiento, integración y despliegue continuos, y monitoreo. Conforme a la sección 8 del caso, quedan fuera las interfaces gráficas, la autenticación avanzada, los pagos y la analítica compleja; los mensajes que el PB describe para el administrador se entregan como cuerpo de la respuesta HTTP.

Este DAS se ve afectado por el PB de Patent Sweep y por los PB de las API con las que se integra (Materiales/Fórmulas, Fabricación y Notificaciones), y a su vez condiciona el plan de pruebas, la definición del pipeline CI/CD y los contratos de integración con esos equipos.

1.2.1 Módulos y funcionalidades

Tabla 1. Módulos y funcionalidades (prioridad y complejidad según la hoja de estimación del PB)

Módulo	HU	Funcionalidad	Prioridad	Complejidad
Barrido automático	HU-01	Automatizar la ejecución del barrido periódico de patentes	Must	20
	HU-12	Generar alertas automáticas ante nuevos conflictos (POST /notificaciones)	Could	8
Búsqueda y evaluación	HU-02	Validar manualmente patentes en conflicto con una fórmula	Must	8
	HU-03	Evaluar una fórmula nueva (invocado por Materiales/Fórmulas)	Should	13
Estado e historial	HU-04	Consultar el estado del barrido de una fórmula (invocado por Fabricación)	Must	5
	HU-05	Consultar el historial de búsquedas y barridos	Should	5
Gestión de conflictos	HU-07	Consultar el listado de conflictos detectados	Must	5
	HU-08	Ver el detalle de una patente en conflicto	Should	3
	HU-09	Descartar manualmente un conflicto	Must	5
Configuración	HU-06	Configurar la frecuencia del barrido	Could	3
	HU-10	Configurar los criterios de búsqueda del barrido	Should	5
	HU-11	Consultar los criterios de búsqueda vigentes	Could	2

1.2.2 Decisiones arquitecturales

Tabla 2. Decisiones arquitecturales

ID	Decisión	Justificación
DA-01	API REST independiente con base de datos propia (PostgreSQL 16).	Enfoque SOA del caso: reemplazar el hub homegrown acoplado por servicios que escalan y evolucionan por separado.

ID	Decisión	Justificación
DA-02	Arquitectura multicapa Controller – Service – Integration – Repository sobre Java 21 y Spring Boot 3.	Separación de responsabilidades; ecosistema con Validation, Data JPA, Scheduling, Actuator y Resilience4j.
DA-03	Barrido con planificador dinámico que lee la frecuencia vigente en cada ciclo.	HU-01 y HU-06: el cambio de frecuencia aplica en la siguiente ejecución sin reiniciar ni interrumpir la actual.
DA-04	Evaluación de fórmulas asíncrona: 202 Accepted inmediato y dictamen por callback a Materiales/Fórmulas.	HU-03 permite hasta 30 s, pero Materiales/Fórmulas espera respuesta en 5 s y define el callback POST /formulas/{id}/dictamen-patente (su HU-10). Ver S-03.
DA-05	Fuente externa de patentes detrás de un puerto (FuentePatentesPort) con adaptador intercambiable.	El PB no define la fuente (HU-02, nota del PO). Se inicia con un dataset local y se puede conectar una fuente real sin tocar la lógica.
DA-06	Resiliencia en integraciones salientes con Resilience4j: Retry (3 intentos, 5 s), TimeLimiter y Circuit Breaker.	HU-01, HU-02, HU-03 y HU-12 exigen 3 reintentos y tiempos máximos de 20 s y 30 s; el Circuit Breaker evita esperar en cada llamada cuando una API está caída.
DA-07	Notificaciones desacopladas del barrido mediante eventos de dominio asíncronos y estado de notificación persistido.	HU-12: no detener ni retrasar el barrido, una sola notificación por conflicto y 3 intentos antes de revisión manual.
DA-08	Estado de fórmula precalculado (modelo de lectura) al cerrar cada revisión.	HU-04: respuesta en 1 s como máximo y sin ejecutar una búsqueda en tiempo real.
DA-09	Catálogo IPC/CPC estático en base de datos con caché en memoria (Caffeine).	HU-02, HU-03 y HU-10 validan clasificaciones en cada petición; el catálogo cambia con poca frecuencia.
DA-10	Historial de revisiones y de configuraciones solo de inserción (append-only), con retención mínima de 12 meses.	HU-05 (trazabilidad y retención), HU-06 y HU-10 (historial de configuraciones con una sola activa).
DA-11	Autenticación con login y token de sesión (JWT) y autorización por roles (RBAC) con Spring Security.	HU-05 y HU-09 requieren registrar el usuario de una sesión autenticada. La sección 8 del caso excluye la autenticación avanzada (OAuth2, SSO, MFA), no la básica. Ver S-06 y S-10.
DA-12	Errores con ProblemDetail (RFC 9457) y traceId.	Todas las HU definen códigos y mensajes exactos; el traceId permite rastrear fallos entre API.

ID	Decisión	Justificación
DA-13	Contenedor Docker en AWS EC2 desplegado por un pipeline con quality gate de SonarCloud; monitoreo con Prometheus y Grafana.	Lineamientos obligatorios de las secciones 6 y 7 del caso.

1.2.3 Supuestos y pendientes de integración

El PB marca varios contratos como pendientes con otros equipos y contiene algunas inconsistencias que afectan la arquitectura. Este DAS adopta los supuestos de la tabla 3, que deben confirmarse antes de implementar las HU afectadas.

Tabla 3. Supuestos y pendientes de integración

ID	Pendiente o inconsistencia	HU	Supuesto adoptado en este DAS	Acordar con
S-01	HU-01 pide a Materiales/Fórmulas un POST con estado y año, y espera año de radicación, título (60) y descripción (500). El PB de Materiales/Fórmulas no tiene ese endpoint ni esos campos (tiene GET /formulas con filtro de estado, nombre de 80 y aplicación técnica de 200).	HU-01	Consumir GET /formulas?estado=APROBADA paginado; mapear nombre → título y aplicación técnica → descripción; el año queda opcional.	Equipo de Materiales/Fórmulas
S-02	Identificador de fórmula: numérico de 6 dígitos (HU-03, HU-07), sin definir (HU-04, HU-05); Materiales/Fórmulas usa códigos «form-NNN».	HU-03 a HU-07	formulaId = código de la fórmula de Materiales/Fórmulas (texto, máx. 20).	Equipos de Materiales/Fórmulas y de Fabricación
S-03	HU-03 responde 200 de forma síncrona hasta en 30 s y usa «sin infracciones / conflicto detectado»; Materiales/Fórmulas espera 5 s, un ticket y un callback con SIN_INFRACCION / INFRACCION. La ruta del PB contiene «ñ» (evaluar-formula).	HU-03	202 Accepted + callback POST /formulas/{id}/dictamen-patente con el vocabulario de Materiales/Fórmulas; ruta /patentes/evaluar-formula.	Equipo de Materiales/Fórmulas

ID	Pendiente o inconsistencia	HU	Supuesto adoptado en este DAS	Acordar con
S-04	La fuente externa de patentes no está definida.	HU-01, HU-02, HU-03	Adaptador sobre un dataset local de patentes; se podrá reemplazar por una fuente real (p. ej., EPO OPS).	Profesor / Equipo de Patent Sweep (original)
S-05	El contrato de POST /notificaciones está pendiente.	HU-12	Cuerpo {tipo: CONFLICTO_PATENTE, formulaId, patenteId, conflictoId, fechaDeteccion}; éxito = 2xx.	Equipo de Notificaciones
S-06	HU-09 toma el administrador «de la sesión autenticada», pero el caso excluye la autenticación avanzada.	HU-05, HU-09	Login básico (POST /auth/login) que emite un JWT; el usuario y su rol se toman del token en todos los endpoints. Sin OAuth2, SSO ni MFA.	Profesor
S-07	HU-02: 20 s de tiempo máximo con 3 reintentos espaciados 5 s ya suman 15 s de espera sin contar las llamadas.	HU-02	Límite global de 20 s (TimeLimiter) que cancela los reintentos restantes y responde 500.	Equipo de Patent Sweep (original)
S-08	HU-11 se titula «consultar las patentes vigentes», pero su criterio consulta los criterios de búsqueda vigentes.	HU-11	Se implementa la consulta de criterios (GET /patentes/configuracion/criterios-busqueda).	Equipo de Patent Sweep (original)
S-09	HU-06 admite unidades HORAS, DIAS y SEMANAS, pero el rango se expresa en horas (1 a 720).	HU-06	Se convierte a horas antes de validar (p. ej., 5 SEMANAS = 840 h → 400).	Equipo de Patent Sweep (original)
S-10	Con autorización por roles, los endpoints que consumen otras API (HU-03 y HU-04) también exigen token.	HU-03, HU-04	Rol SERVICIO con un token por API consumidora, emitido por Patent Sweep.	Equipos de Materiales/Fórmulas y de Fabricación

1.2.4 Alcance del documento

- **Atributos de calidad** observables y no observables, sección 2.1.
- **Patrones de diseño:** sección 2.2.
- **Atributos vs. patrones:** sección 2.3.
- **Representación de la solución mediante vistas UML (Kruchten):** secciones 3 (vista física), 4 (vista lógica) y 5 (vista de despliegue).

1.3 Definiciones, siglas y abreviaturas

Tabla 4. Definiciones, siglas y abreviaturas

Término	Definición
API REST	Interfaz de programación expuesta sobre HTTP que intercambia recursos en JSON con verbos estándar (GET, POST, PUT, PATCH).
SOA	Arquitectura orientada a servicios: el sistema se compone de servicios independientes que se comunican por contratos.
DAS	Documento de Arquitectura de Software.
PB / HU	Product Backlog / Historia de Usuario.
MoSCoW	Técnica de priorización: Must, Should, Could, Won't.
Barrido	Ejecución programada que revisa todas las fórmulas activas contra la fuente de patentes (HU-01).
Fórmula	Fórmula de cuchilla registrada en Materiales/Fórmulas y vigilada por Patent Sweep.
Conflicto	Coincidencia entre una fórmula y una patente vigente; puede estar DETECTADO o DESCARTADO.
Dictamen	Resultado de la evaluación legal de una fórmula: sin infracciones o conflicto detectado.
IPC / CPC	Clasificación Internacional de Patentes (OMPI) y Clasificación Cooperativa de Patentes (EPO/USPTO).
Descarte	Acción del administrador que marca un conflicto como falso positivo (HU-09).
DTO	Data Transfer Object: objeto que define el contrato de entrada o salida de un endpoint.
JPA / ORM	Jakarta Persistence API / mapeo objeto-relacional (Hibernate).
IoC / DI	Inversión de control e inyección de dependencias (contenedor de Spring).
Resilience4j	Biblioteca de tolerancia a fallos para Java (Retry, TimeLimiter, Circuit Breaker).
Callback	Llamada de retorno: el servicio invocado notifica el resultado al invocador en una petición posterior.
ProblemDetail	Formato estándar de errores HTTP en JSON definido por el RFC 9457.
traceId	Identificador de correlación que acompaña una petición en logs y respuestas de error.
Append-only	Registro en el que solo se insertan datos; no se actualizan ni se eliminan antes de su retención.
CI/CD	Integración continua / despliegue continuo.

Término	Definición
PR	Pull Request: solicitud de integración de cambios revisada antes del merge.
Quality gate	Conjunto de condiciones de SonarCloud que el código debe cumplir para desplegarse.
AWS EC2	Servicio de máquinas virtuales de Amazon Web Services.
Docker / Compose	Plataforma de contenedores / herramienta para definir y ejecutar varios contenedores.
Prometheus / Grafana	Recolector de métricas por pull / herramienta de visualización y tableros.
Actuator / Micrometer	Endpoints operativos de Spring Boot / fachada de métricas de la JVM y la aplicación.
OpenAPI / Swagger	Especificación del contrato REST / interfaz web para explorarlo y probarlo.
JWT	JSON Web Token: token de sesión firmado que transporta la identidad y el rol del usuario (RFC 7519).
RBAC	Role-Based Access Control: control de acceso en el que los permisos se asignan a roles y los roles a usuarios.
Health check	Endpoint que informa si la aplicación y sus dependencias están operativas (en Spring Boot, /actuator/health).

1.4 Referencias

Tabla 5. Referencias

Documento	Versión	Fecha de la versión
Product Backlog Patent Sweep (20261015_PB_Patent_Sweep_V3.0)	3.0	15/10/2026
Product Backlog Materiales/Fórmulas (contrato de integración, HU-09 y HU-10)	1.5	15/10/2026
Caso de estudio: Plataforma de automatización de cadena de suministro orientada a servicios — Cutit Saws Ltd.	—	2026-2
Caso Cutit 1 (Contexto del negocio), 2 (Procesos y operación) y 3 (Problemas y necesidades)	—	2026-2
Plantilla DAS — Arquitectura Light	—	—
ISO/IEC 25010:2023 Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model	2023	11/2023

Documento	Versión	Fecha de la versión
Kruchten, P. «Architectural Blueprints — The 4+1 View Model of Software Architecture». IEEE Software 12(6)	—	11/1995
Bass, L., Clements, P. y Kazman, R. Software Architecture in Practice. Addison-Wesley	4.ª ed.	2021
RFC 9457 Problem Details for HTTP APIs (IETF)	—	07/2023

1.5 Vista Global

El resto del documento describe la solución de lo general a lo particular. La sección 2 ubica a Patent Sweep dentro de la plataforma y define sus capas; la sección 2.1 fija los atributos de calidad y las secciones 2.2 y 2.3 los patrones que los materializan. Las secciones 3 a 5 representan la solución con vistas UML. La API se organiza en las siguientes capas:

- **Capa de presentación (interfaz REST):** controladores y DTO JSON. No hay interfaz gráfica.
- **Capa de lógica de negocio:** servicios, planificador del barrido y motor de coincidencias.
- **Capa de integración:** clientes de Materiales/Fórmulas y Notificaciones, y puerto de la fuente de patentes.
- **Capa de acceso a datos:** repositorios Spring Data JPA y entidades sobre PostgreSQL.
- **Capa transversal (utility):** validación, manejo de errores, seguridad, resiliencia, caché, documentación y métricas.

Correspondencia con el modelo 4+1: la «vista física» de la plantilla (sección 3) muestra los componentes de software, que en Kruchten corresponden a la vista de desarrollo; la «vista de despliegue» (sección 5) corresponde a la vista física de Kruchten; la vista lógica y de procesos se presenta en la sección 4, y los escenarios son las HUs que recorren los diagramas de secuencia.

2. Macro Arquitectura

La plataforma de Cutit Saws Ltd. está compuesta por cinco API REST independientes (Materiales/Fórmulas, Patent Sweep, Fabricación, Inventario y Notificaciones), cada una con su propia base de datos y su propio despliegue. Se integran punto a punto mediante HTTP/JSON; cada API consume al menos una API de otro equipo y es consumida por al menos una distinta (sección 5 del caso). Particularmente, para la división de Patent Sweep, y a modo de acercamiento al flujo que tendrá con los demás departamentos, se logran identificar las relaciones:

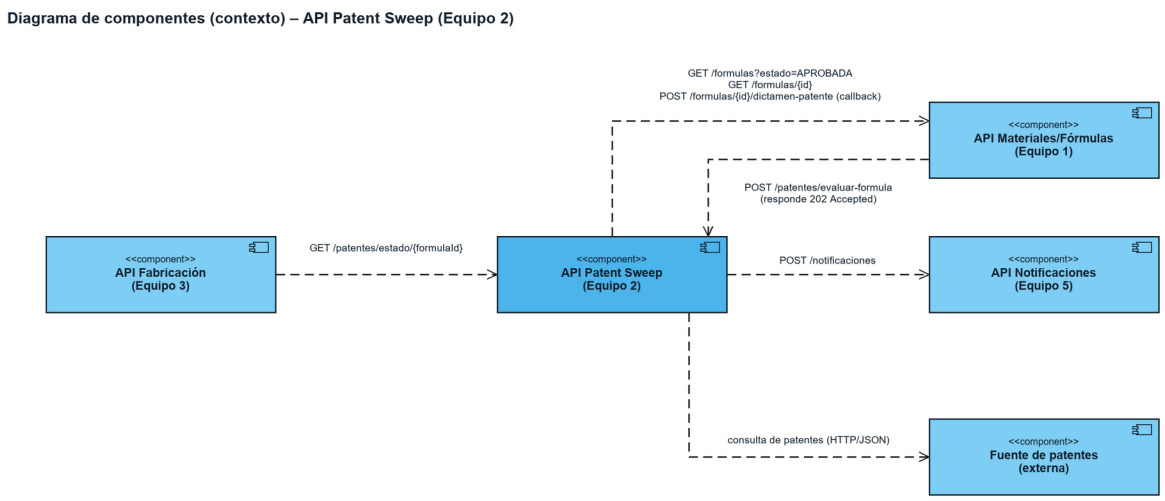


Diagrama 1. Contexto de integración de la API Patent Sweep en la plataforma SOA de Cutit Saws.

En su relación con el servicio de Materiales/Fórmulas, Patent Sweep interactúa de manera bidireccional. Por un lado, Patent Sweep consume los recursos expuestos por Materiales para listar el catálogo de fórmulas aprobadas e inspeccionar el detalle técnico de sus componentes para las revisiones automáticas periódicas. Por otro lado, cuando en el módulo de Materiales se solicita evaluar la infracción de patentes para una fórmula específica, éste invoca a Patent Sweep para procesar dicha solicitud de forma asíncrona. Una vez determinado el resultado del análisis, Patent Sweep notifica la conclusión de vuelta al servicio de Materiales enviando el dictamen técnico correspondiente.

Respecto a la integración con el servicio de Fabricación, Patent Sweep actúa como un servicio proveedor de información técnica de control. La API de Fabricación consume el endpoint expuesto por Patent Sweep para consultar el estado de barrido y las alertas vigentes de cualquier fórmula. Esta verificación le permite al sistema de producción confirmar en tiempo real si una fórmula se encuentra libre de riesgos legales antes de autorizar su procesamiento en planta.

Por su parte, en la interacción idónea con el servicio de Notificaciones, Patent Sweep adopta un rol de cliente consumidor. Ante la detección de un conflicto potencial entre una fórmula y una patente registrada, Patent Sweep envía una petición al API de Notificaciones para solicitar el despacho automático de las alertas dirigidas a los responsables del área correspondiente, garantizando así la comunicación inmediata de hallazgos.

Para la verificación de propiedad intelectual, Patent Sweep se comunica con la Fuente Externa de Patentes operando como un cliente que realiza consultas HTTP/JSON hacia este proveedor externo. A través de este enlace, el sistema envía búsquedas basadas en los componentes de las fórmulas, términos técnicos, clasificaciones internacionales de patentes y rangos de fechas, obteniendo la información necesaria para determinar coincidencias y potenciales infracciones.

Finalmente, la API Patent Sweep expone una serie de contratos RESTful orientados al consumo directo por parte de la interfaz gráfica de usuario. A través de estos servicios, la

aplicación cliente permite la ejecución de búsquedas manuales bajo demanda, la inspección del historial de revisiones registradas, la gestión y descarte justificado de alertas de conflicto, y la configuración administrativa de las reglas de búsqueda y la frecuencia de las ejecuciones programadas. Internamente, el componente estructura su código mediante una arquitectura multicapa, para garantizar una clara separación entre la gestión de peticiones HTTP, la lógica de negocio, las integraciones externas y el acceso a datos.

Internamente, la API sigue una arquitectura multicapa. La tabla 6 resume las responsabilidades de cada capa y cómo se comunican.

Tabla 6. Capas de la API Patent Sweep

Capa	Componentes	Responsabilidad	Comunicación
Presentación (REST)	BusquedaController, EvaluacionController, EstadoFormulaController, HistorialController, ConflictoController, ConfiguracionController; DTO	Recibir HTTP, validar la forma de la petición, traducir a DTO y delegar. No contiene reglas de negocio.	HTTP/JSON con clientes; invoca servicios por inyección de dependencias.
Lógica de negocio	BarridoScheduler, BarridoService, BusquedaService, EvaluacionService, MotorCoincidencias, ConflictoService, EstadoFormulaService, ConfiguracionService, NotificacionService	Reglas de negocio, transiciones de estado, límites transaccionales, planificación del barrido y emisión de eventos.	Llamadas a métodos Java; eventos de dominio asíncronos; usa repositorios y clientes.
Integración	MaterialesClient, NotificacionesClient, FuentePatentesPort + adaptador	Encapsular las API externas, aplicar reintentos y tiempos máximos, y traducir contratos externos al modelo propio.	HTTP/JSON saliente (RestClient) protegido con Resilience4j.
Acceso a datos	Repositorios Spring Data JPA, entidades, Hibernate, HikariCP	Persistencia, consultas paginadas y filtradas, historial append-only.	JDBC hacia PostgreSQL.
Transversal (utility)	Validator, GlobalExceptionHandler, Spring Security, Resilience4j, Caffeine, springdoc, Lombok, Actuator/Micrometer	Aspectos comunes a todas las capas.	Anotaciones y configuración de Spring.

Reglas de dependencia: cada capa sólo depende de la inmediatamente inferior o de la capa transversal; los controladores no acceden a repositorios; las entidades no salen de la capa de lógica (se exponen DTO), y toda llamada a otra API pasa por la capa de integración.

2.1 Metas y Restricciones Arquitectónicas

En las tablas siguientes, cada patrón se identifica con su ID de la tabla 7 (sección 2.2), junto con la definición que la norma ISO/IEC 25010:2023 propone para cada atributo de calidad y una definición breve de lo que hace; cada patrón citado para un atributo está marcado en la característica correspondiente de la matriz de la tabla 8 (sección 2.3). Con el criterio del curso, un atributo es observable cuando su solución se puede señalar en el código (capa, clase, anotación o archivo) y no observable cuando sólo se evidencia en el comportamiento de la API; por eso algunos atributos aparecen en ambas tablas, cada uno con su parte.

1. Atributos observables

Atributo de Calidad	Descripción	Tácticas / Patrón de Diseño	Dónde se aplica
Adecuación funcional: • Corrección funcional	Capacidad del producto o sistema para proveer resultados exactos cuando es usado por los usuarios especificados.	**P06 Validator:** las restricciones se declaran en los DTO de entrada y también en las entidades del modelo (Bean Validation sobre JPA): un valor obligatorio que no llega o un valor no permitido hace fallar la operación de inmediato. **P17 State (máquina de estados):** cada entidad tiene estados definidos y solo se permiten las transiciones válidas entre ellos.	Controllers Layer: los DTO de entrada de BusquedaController y EvaluacionController llevan @NotNull, @Size(max = 20) y @Pattern sobre el identificador del diseño y los códigos IPC/CPC, y los controladores los reciben con @Valid. Data Access Layer: las entidades del Model repiten las restricciones con @Column(nullable = false, length = …). Logic Layer: ConflictoService solo cambia el estado de un conflicto con las transiciones DETECTADO ↔ DESCARTADO definidas en su enum de estados.

Eficiencia de desempeño: • Comportamiento temporal	Grado en que un producto realiza sus funciones de forma que el tiempo de respuesta y el ratio de rendimiento cumple los requisitos especificados.	**P11 Solicitud-respuesta asíncrona (callback):** responde de inmediato que la solicitud fue aceptada, procesa en segundo plano (@Async) y entrega el resultado después en otra petición. **P16 Modelo de lectura precalculado:** el resultado que más se consulta se calcula al escribir y se guarda listo para leer. **P15 Cache-aside:** se consulta primero la caché; si el dato no está, se lee de la base de datos y se guarda en caché.	Logic Layer: EvaluacionService ejecuta la evaluación con @Async sobre un ThreadPoolTaskExecutor propio y EvaluacionController devuelve ResponseEntity.accepted () (202). EstadoFormulaService lee la entidad de estado precalculado en lugar de consultar la fuente de patentes. Utility Layer: el catálogo IPC/CPC se lee con @Cacheable sobre Caffeine.

Eficiencia de desempeño: • Utilización de recursos	Grado en que la cantidad y tipos de recursos utilizados por el producto al llevar a cabo su función bajo condiciones determinadas no exceden lo especificado.	**P03 Inyección de dependencias** el contenedor de Spring crea cada componente una sola vez (singleton) y lo comparte entre todas las peticiones, en lugar de instanciar objetos nuevos en cada una; los que solo usan algunos flujos pueden declararse @Lazy para crearse cuando se usan por primera vez. **P12 Eventos de dominio / Observer asíncrono:** quien detecta un hecho publica un evento y los interesados lo procesan aparte (@Async), sin bloquear al emisor. **P20 Paginación:** devuelve los resultados por bloques de tamaño fijo en lugar de todos a la vez. **P15 Cache-aside:** se consulta primero la caché; si el dato no está, se lee de la base de datos y se guarda en caché.	Todas las capas: dependencias inyectadas por constructor en clases @RestController, @Service y @Repository, que el contenedor crea una sola vez. NotificacionService escucha ConflictoDetectadoEvent con @TransactionalEventListener y @Async. Data Access Layer e Integration Layer: los repositorios reciben un Pageable de 20 elementos y MaterialesClient pide /formulas por páginas de 50.
Eficiencia de desempeño: • Capacidad	Grado en que el producto cumple los requisitos relativos a límites máximos para un parámetro (ítems almacenados, usuarios concurrentes, ancho de banda de comunicaciones...).	**P20 Paginación:** devuelve los resultados por bloques de tamaño fijo en lugar de todos a la vez. **P06 Validator:** verifica obligatoriedad, formato y rangos (@NotNull, @Size, @Pattern) antes de procesar; si un dato no cumple, la petición falla de inmediato con 400.	Tamaños de página fijos como constantes en los controladores y en MaterialesClient (20 y 50); tamaños máximos de campo con @Size en los DTO y length en @Column; maximum-pool-size de HikariCP en application.yml.

Compatibilidad: • Coexistencia	Capacidad del producto para coexistir con otro software independiente, en un entorno común, compartiendo recursos comunes sin detrimento.	**P02 SOA / API REST:** cada capacidad de negocio es un servicio independiente con su propio contrato y su propia base de datos. **P22 Contenerización + pipeline CI/CD:** la API y sus dependencias se empaquetan en una imagen que se construye, prueba y despliega automáticamente.	docker-compose.yml: servicios api-patent-sweep, postgres, prometheus, grafana y node-exporter en una red propia de Docker; solo se publican 8080 y 3000. application.yml apunta a la base de datos propia de Patent Sweep (DA-01).
Compatibilidad: • Interoperabilidad	Capacidad de dos o más sistemas o componentes para intercambiar información y utilizar la información intercambiada.	**P09 Adapter / Gateway:** traduce la interfaz de un sistema externo a una interfaz propia, para que la lógica no dependa de él. **P05 DTO + Mapper:** objeto de transferencia que expone entre capas y hacia el cliente solo los campos necesarios, no la entidad completa. **P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código.	Integration Layer: MaterialesClient y NotificacionesClient (RestClient) traducen los DTO externos al modelo propio (nombre → título, aplicación técnica → descripción; SIN_INFRACCION / INFRACCION, S-01 y S-03). FuentePatentesPort es una interfaz con un adaptador. springdoc-openapi publica el contrato en /v3/api-docs.

Capacidad de interacción: • Operabilidad	Capacidad del producto que permite al usuario operarlo y controlarlo con facilidad.	**P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código. **P20 Paginación:** devuelve los resultados por bloques de tamaño fijo en lugar de todos a la vez. **P07 Manejador global de excepciones:** un único @RestControllerAdvice captura las excepciones que suben desde cualquier capa y las convierte en una respuesta de error uniforme.	Swagger UI expuesto por springdoc en /swagger-ui.html. ConflictoController recibe estado, formulaId y page como @RequestParam (HU-07). GlobalExceptionHandler (@RestControllerAdvice) devuelve ProblemDetail con el mensaje de cada criterio.
Capacidad de interacción: • Protección contra errores	Capacidad del sistema para prevenir errores en su operación.	**P06 Validator:** el mismo validador de los DTO y del modelo (ver Corrección funcional) impide que datos incorrectos entren a la transacción. **P13 Idempotencia:** repetir la misma operación produce el mismo resultado que ejecutarla una sola vez. **P17 State (máquina de estados):** cada entidad tiene estados definidos y solo se permiten las transiciones válidas entre ellos.	ConfiguracionService convierte HORAS, DIAS y SEMANAS a horas y valida el rango 1–720 antes de guardar (S-09); un validador de catálogo verifica cada código IPC/CPC (HU-02, HU-10). ConflictoService responde 409 si el conflicto ya está DESCARTADO, y ConflictoController expone PATCH /patentes/conflictos/{conflictoId}/reactivacion para deshacer un descarte (HU-09).

Capacidad de interacción: • Asistencia al usuario	Capacidad del producto que permite que sea utilizado por usuarios con determinadas características y capacidades logrando objetivos específicos.	**P24 Contrato OpenAPI:** cada endpoint, parámetro y campo lleva summary, description y example en la especificación (@Operation, @Parameter, @Schema), que Swagger UI muestra al usuario mientras arma la petición. **P07 Manejador global de excepciones:** un único @RestControllerAdvice captura las excepciones que suben desde cualquier capa y las convierte en una respuesta de error uniforme.	Como la API no tiene interfaz gráfica (sección 8 del caso), la asistencia está en el código de la documentación y de los errores: springdoc-openapi con @Operation y @Schema(example) en cada controlador y DTO, para que el administrador o un integrante de otro equipo complete su tarea desde Swagger UI sin conocer el código. GlobalExceptionHandler devuelve en el campo detail del ProblemDetail el mensaje en español que define cada HU e indica qué corregir (p. ej., la clasificación IPC inválida o el rango de frecuencia permitido).
Capacidad de interacción: • Auto-descriptividad	Capacidad de un producto para presentar la información adecuada, haciendo su uso inmediatamente evidente para el usuario sin interacciones excesivas con el producto u otros recursos (documentación, help desks, otros usuarios...).	**P24 Contrato OpenAPI:** cada campo, parámetro y operación lleva description y example en la especificación (@Schema, @Parameter, @Operation), que Swagger UI muestra junto al campo: es el equivalente del tooltip en una API sin interfaz gráfica. **P07 Manejador global de excepciones:** un único @RestControllerAdvice captura las excepciones que suben desde cualquier capa y las convierte en una respuesta de error uniforme.	Controllers Layer: @Operation(summary, description) en cada endpoint, @Parameter en sus parámetros y @Schema(description, example) en los campos de los DTO. GlobalExceptionHandler arma cada ProblemDetail con title, detail y traceId.

Fiabilidad: • Ausencia de fallos	Capacidad del sistema de llevar a cabo sus funciones sin fallos bajo condiciones normales de operación.	**P07 Manejador global de excepciones:** las excepciones se capturan (try/catch) y se manejan en lugar de dejar que tumben la petición o el servicio; las que no se resuelven localmente suben a un único @RestControllerAdvice que las convierte en ProblemDetail. **P22 Contenerización + pipeline CI/CD:** la API y sus dependencias se empaquetan en una imagen que se construye, prueba y despliega automáticamente.	Logic Layer: try/catch en BarridoService alrededor del procesamiento de cada fórmula. Integration Layer: try/catch en los clientes para traducir errores HTTP a excepciones propias. Utility Layer: GlobalExceptionHandler con un @ExceptionHandler por tipo de excepción. .github/workflows/ci-cd.yml ejecuta mvn verify y el análisis de SonarCloud, y solo despliega si el quality gate se aprueba (DA-13).
Fiabilidad: • Disponibilidad	Capacidad del sistema o componente de estar operativo y accesible para su uso cuando se requiere.	**P28 Health Check:** un endpoint informa si la API y sus dependencias están operativas; quien la ejecuta lo consulta periódicamente y actúa si falla. **P25 Circuit Breaker:** tras varios fallos seguidos, deja de llamar a la dependencia durante un intervalo y falla de inmediato; después prueba de nuevo.	application.yml habilita /actuator/health con el indicador de PostgreSQL. docker-compose.yml define para api-patent-sweep un healthcheck contra ese endpoint (interval, retries) y restart: unless-stopped, que reinicia el contenedor si el proceso termina; para reiniciarlo también cuando queda unhealthy se añade un monitor (p. ej., autoheal), porque Docker Engine por sí solo no lo hace. Integration Layer: @CircuitBreaker sobre los métodos de los clientes, configurado en application.yml.

Fiabilidad: • Tolerancia a fallos	Capacidad del sistema o componente para operar según lo previsto en presencia de fallos hardware o software.	**P10 Retry + Timeout:** repite una llamada que falló por una causa transitoria y corta la espera al superar un tiempo máximo. **P25 Circuit Breaker:** tras varios fallos seguidos, deja de llamar a la dependencia durante un intervalo y falla de inmediato; después prueba de nuevo. **P14 Aislamiento de fallos (bulkhead lógico):** el fallo se captura con try/catch en la unidad donde ocurre (cada fórmula) y no se propaga al resto del proceso. **P07 Manejador global de excepciones:** un único @RestControllerAdvice captura las excepciones que suben desde cualquier capa y las convierte en una respuesta de error uniforme.	Integration Layer: @Retry y @TimeLimiter (sobre métodos que devuelven CompletableFuture) en MaterialesClient, NotificacionesClient y el adaptador de FuentePatentesPort, con 3 intentos, 5 s de espera y límites de 20 s y 30 s en application.yml; @CircuitBreaker sobre los mismos métodos. Logic Layer: try/catch por fórmula en BarridoService, que marca NO_PROCESADO el que falla y continúa con el siguiente.
Fiabilidad: • Capacidad de recuperación	Capacidad del producto software para recuperar los datos directamente afectados y reestablecer el estado deseado del sistema en caso de interrupción o fallo.	**P28 Health Check:** un endpoint informa si la API y sus dependencias están operativas; quien la ejecuta lo consulta periódicamente y actúa si falla. **P13 Idempotencia:** repetir la misma operación produce el mismo resultado que ejecutarla una sola vez.	Data Access Layer: entidad de notificación con su estado y restricción única por conflicto, que impide reenviar una alerta ya enviada; entidad de barrido con estado EN_CURSO, COMPLETADO o FALLIDO. docker-compose.yml: healthcheck y restart para api-patent-sweep, y volumen persistente para postgres.

Seguridad: • Confidencialidad	Capacidad de asegurar que los datos sólo son accesibles a aquellos con autorización para ello.	**P27 Control de acceso basado en roles (RBAC):** cada usuario tiene un rol y cada operación declara qué roles pueden ejecutarla. **P05 DTO + Mapper:** objeto de transferencia que protege la información que viaja entre capas: expone solo lo necesario y no el objeto completo. **P21 Configuración externalizada:** los valores que cambian entre entornos se leen de variables de entorno y perfiles, no del código.	Utility Layer: SecurityFilterChain con authorizeHttpRequests por ruta y rol. Controllers Layer: los controladores devuelven DTO, nunca entidades. application.yml lee las credenciales y la clave de firma de los JWT de variables de entorno (p. ej., ${DB_PASSWORD}, ${JWT_SECRET}), que ci-cd.yml toma de los secrets de GitHub.
Seguridad: • Integridad	Capacidad de un producto para garantizar que el estado de su sistema y sus datos están protegidos frente a modificaciones o eliminaciones no autorizadas, ya sea por acciones malintencionadas o por errores informáticos.	**P27 Control de acceso basado en roles (RBAC):** cada usuario tiene un rol y cada operación declara qué roles pueden ejecutarla. **P18 Registro de auditoría append-only:** los registros solo se insertan; no se actualizan ni se borran durante su retención. **P17 State (máquina de estados):** cada entidad tiene estados definidos y solo se permiten las transiciones válidas entre ellos. **P06 Validator:** verifica obligatoriedad, formato y rangos (@NotNull, @Size, @Pattern) antes de procesar; si un dato no cumple, la petición falla de inmediato con 400.	@PreAuthorize("hasRole ('ADMINISTRADOR')") en los PUT de ConfiguracionController y en los PATCH de ConflictoController. @Transactional en ConflictoService y @TransactionalEventListener(phase = AFTER_COMMIT) para ConflictoDetectadoEvent. Data Access Layer: los repositorios de historial solo insertan (save); no exponen actualización ni borrado (DA-10).

Seguridad: • No repudio	Capacidad de demostrar las acciones o eventos que han tenido lugar, de manera que dichas acciones o eventos no puedan ser repudiados posteriormente.	**P26 Autenticador + autorizador (token de sesión):** el autenticador verifica la identidad en el login y emite un token de sesión firmado; el autorizador valida ese token en cada petición antes de llegar al controlador. **P18 Registro de auditoría append-only:** los registros solo se insertan; no se actualizan ni se borran durante su retención. **P19 Correlation ID (traceId):** un identificador único acompaña cada petición en logs, respuestas de error y llamadas a otras API.	Las entidades de historial guardan el usuario autenticado (tomado del SecurityContextHolder), la fecha, los parámetros y el traceId; las escriben ConflictoService, ConfiguracionService y BarridoService (HU-05, HU-06, HU-09, HU-10).
Seguridad: • Responsabilidad	Capacidad de rastrear de forma inequívoca las acciones de una entidad.	**P18 Registro de auditoría append-only:** los registros solo se insertan; no se actualizan ni se borran durante su retención. **P19 Correlation ID (traceId):** un identificador único acompaña cada petición en logs, respuestas de error y llamadas a otras API.	Micrometer Tracing pone el traceId en el MDC; el patrón de logs de application.yml lo imprime (%X{traceId}) y GlobalExceptionHandler lo agrega a cada ProblemDetail.

Seguridad: • Autenticidad	Capacidad de un producto para demostrar que la identidad de un sujeto o recurso es la que se afirma.	**P26 Autenticador + autorizador (token de sesión):** el autenticador verifica la identidad en el login y emite un token de sesión firmado; el autorizador valida ese token en cada petición antes de llegar al controlador. **P27 Control de acceso basado en roles (RBAC):** cada usuario tiene un rol y cada operación declara qué roles pueden ejecutarla.	Autenticador: AuthController (Controllers Layer) expone POST /auth/login y usa el AuthenticationManager de Spring Security para validar las credenciales y emitir el JWT. Autorizador: un filtro registrado en SecurityFilterChain (Utility Layer) valida firma y vigencia del token antes de llegar a cualquier controlador. Roles ADMINISTRADOR y SERVICIO (S-10). Queda fuera la autenticación avanzada (OAuth2, SSO, MFA) que excluye la sección 8 del caso (DA-11, S-06).
Seguridad: • Resistencia	Capacidad de mantener la operación de un producto bajo condiciones de ataque de un actor malicioso.	**P06 Validator:** verifica obligatoriedad, formato y rangos (@NotNull, @Size, @Pattern) antes de procesar; si un dato no cumple, la petición falla de inmediato con 400. **P20 Paginación:** devuelve los resultados por bloques de tamaño fijo en lugar de todos a la vez. **P10 Retry + Timeout:** repite una llamada que falló por una causa transitoria y corta la espera al superar un tiempo máximo.	@Valid en todos los @RequestBody; el tamaño de página está fijado en el código y no se toma del cliente; @TimeLimiter en los clientes. Puertos publicados limitados en docker-compose.yml y en el Security Group de la EC2 (22, 8080 y 3000).

Mantenibilidad: • Modularidad	Capacidad de un producto para evitar que los cambios en un componente afecten a otros componentes.	**P01 Arquitectura multicapa:** organiza el código en capas con responsabilidades separadas y dependencias en un solo sentido. **P09 Adapter / Gateway:** traduce la interfaz de un sistema externo a una interfaz propia, para que la lógica no dependa de él.	Backend: el proyecto se organiza en Controllers, Logic, Integration, Data Access y Utility Layer (figura 2) con las reglas de dependencia de la tabla 6; FuentePatentesPort es una interfaz de la Integration Layer. Frontend: no hace parte del alcance (sección 8 del caso); la API la consumen Swagger UI, Postman y las otras API.
Mantenibilidad: • Reusabilidad	Capacidad de un activo que permite que sea utilizado en más de un sistema software o en la construcción de otros activos.	**P09 Adapter / Gateway:** traduce la interfaz de un sistema externo a una interfaz propia, para que la lógica no dependa de él. **P07 Manejador global de excepciones:** un único @RestControllerAdvice captura las excepciones que suben desde cualquier capa y las convierte en una respuesta de error uniforme. **P05 DTO + Mapper:** objeto de transferencia que expone entre capas y hacia el cliente solo los campos necesarios, no la entidad completa.	MotorCoincidencias (Logic Layer) lo invocan BarridoService, BusquedaService y EvaluacionService, y los tres usan FuentePatentesPort. GlobalExceptionHandler, los mappers y la configuración de Resilience4j en application.yml son comunes a todos los endpoints y clientes.

Mantenibilidad: • Analizabilidad	Facilidad con la que se puede evaluar el impacto de un determinado cambio sobre el resto del software, diagnosticar las deficiencias o causas de fallos en el software, o identificar las partes a modificar.	**P19 Correlation ID (traceId):** un identificador único acompaña cada petición en logs, respuestas de error y llamadas a otras API. **P23 Métricas y monitoreo:** la API expone métricas que un recolector consulta periódicamente y un tablero muestra. **P07 Manejador global de excepciones:** un único @RestControllerAdvice captura las excepciones que suben desde cualquier capa y las convierte en una respuesta de error uniforme. **P18 Registro de auditoría append-only:** los registros solo se insertan; no se actualizan ni se borran durante su retención.	traceId en el MDC y en ProblemDetail; spring-boot-starter-actuator y micrometer-registry-prometheus en el pom.xml exponen /actuator/prometheus; la entidad de barrido registra resultado y conteos (HU-01).

Mantenibilidad: • Capacidad para ser modificado	Capacidad del producto que permite que sea modificado de forma efectiva y eficiente sin introducir defectos o degradar su calidad.	**P01 Arquitectura multicapa:** organiza el código en capas con responsabilidades separadas y dependencias en un solo sentido. **P09 Adapter / Gateway:** traduce la interfaz de un sistema externo a una interfaz propia, para que la lógica no dependa de él. **P22 Contenerización + pipeline CI/CD:** la API y sus dependencias se empaquetan en una imagen que se construye, prueba y despliega automáticamente.	Cambiar la fuente de patentes (p. ej., a EPO OPS) es agregar otra clase que implemente FuentePatentesPort y seleccionarla por configuración, sin tocar MotorCoincidencias ni los controladores (DA-05). El paso que exige el quality gate de SonarCloud está en ci-cd.yml.
Mantenibilidad: • Capacidad para ser probado	Facilidad con la que se pueden establecer criterios de prueba para un sistema o componente y con la que se pueden llevar a cabo las pruebas para determinar si se cumplen dichos criterios.	**P03 Inyección de dependencias (IoC):** el contenedor de Spring crea los objetos y los entrega a quien los necesita, en lugar de que cada clase los instancie. **P09 Adapter / Gateway:** traduce la interfaz de un sistema externo a una interfaz propia, para que la lógica no dependa de él. **P22 Contenerización + pipeline CI/CD:** la API y sus dependencias se empaquetan en una imagen que se construye, prueba y despliega automáticamente. **P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código.	src/test/java: pruebas unitarias de BarridoService, EvaluacionService y ConflictoService con Mockito (@Mock, @InjectMocks), posibles gracias a la inyección por constructor; el adaptador del dataset local sirve como fuente de prueba. ci-cd.yml ejecuta mvn verify en cada PR.

Flexibilidad: • Escalabilidad	Capacidad del producto para gestionar cargas de trabajo crecientes o decrecientes y para adaptar su capacidad a la variabilidad.	**P02 SOA / API REST:** cada capacidad de negocio es un servicio independiente con su propio contrato y su propia base de datos. **P22 Contenerización + pipeline CI/CD:** la API y sus dependencias se empaquetan en una imagen que se construye, prueba y despliega automáticamente.	El contenedor de la API no guarda estado en disco (los datos van a postgres). Si se ejecutan varias réplicas, @SchedulerLock de ShedLock sobre BarridoScheduler evita que el barrido se ejecute dos veces (tabla 14).
Flexibilidad: • Instalabilidad	Facilidad con la que el producto se puede instalar y/o desinstalar de forma exitosa en un determinado entorno.	**P22 Contenerización + pipeline CI/CD:** la API y sus dependencias se empaquetan en una imagen que se construye, prueba y despliega automáticamente. **P21 Configuración externalizada:** los valores que cambian entre entornos se leen de variables de entorno y perfiles, no del código.	Dockerfile multi-stage (compilación con Maven y ejecución con JRE 21), docker-compose.yml con los cinco servicios y .github/workflows/ci-cd.yml, que construye, analiza, publica en Docker Hub y despliega por SSH en la EC2. Perfiles application-dev.yml y application-prod.yml.
Flexibilidad: • Reemplazabilidad	Capacidad del producto para ser utilizado en lugar de otro producto software determinado con el mismo propósito y en el mismo entorno.	**P09 Adapter / Gateway:** traduce la interfaz de un sistema externo a una interfaz propia, para que la lógica no dependa de él. **P04 Repository / DAO:** encapsula el acceso a datos detrás de una interfaz; Spring Data JPA genera la implementación. **P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código.	FuentePatentesPort y los repositorios (interfaces que extienden JpaRepository) aíslan la fuente de patentes y el motor de base de datos. El contrato OpenAPI generado (/v3/api-docs) es lo que otra implementación de Patent Sweep tendría que cumplir.

Protección: • Restricción operativa	Capacidad de un producto para limitar su funcionamiento a unos parámetros o estados seguros cuando se enfrenta a un peligro operativo.	**P17 State (máquina de estados):** cada entidad tiene estados definidos y solo se permiten las transiciones válidas entre ellos. **P16 Modelo de lectura precalculado:** el resultado que más se consulta se calcula al escribir y se guarda listo para leer. **P06 Validator:** verifica obligatoriedad, formato y rangos (@NotNull, @Size, @Pattern) antes de procesar; si un dato no cumple, la petición falla de inmediato con 400.	El peligro es para la propiedad: fabricar una fórmula que infringe una patente vigente. La restricción de construcción queda marcada en la entidad de estado de la fórmula (con conflictos vigentes, sin conflictos o NO_PROCESADO), que EstadoFormulaController expone a Fabricación (HU-04). MaterialesClient solo consulta /formulas?estado=APROBADA (HU-01) y envía el dictamen SIN_INFRACCION / INFRACCION para que Materiales/Fórmulas lo persista (HU-03).
Protección: • Protección ante fallos	Capacidad de un producto para ponerse automáticamente en un modo de funcionamiento seguro o para volver a una condición segura en caso de fallo.	**P14 Aislamiento de fallos (bulkhead lógico):** el fallo se captura con try/catch en la unidad donde ocurre (cada fórmula) y no se propaga al resto del proceso. **P17 State (máquina de estados):** cada entidad tiene estados definidos y solo se permiten las transiciones válidas entre ellos.	En BarridoService, el catch de cada fórmula asigna NO_PROCESADO y nunca SIN_INFRACCION; el barrido pasa a COMPLETADO solo si termina sin excepción, y a FALLIDO en el catch general.
Protección: • Advertencia de peligro	Capacidad de un producto para alertar de riesgos inaceptables, de modo que puedan reaccionar con tiempo suficiente para mantener la seguridad de las operaciones.	**P12 Eventos de dominio / Observer asíncrono:** quien detecta un hecho publica un evento y los interesados lo procesan aparte (@Async), sin bloquear al emisor.	BarridoService publica ConflictoDetectadoEvent con ApplicationEventPublisher; NotificacionService lo procesa con @Async y llama a NotificacionesClient (POST /notificaciones), con @Retry de 3 intentos antes de dejarlo para revisión manual (HU-12).

Protección: • Integración segura	Capacidad de un producto para mantener la seguridad durante y después de la integración con uno o varios componentes	**P09 Adapter / Gateway:** traduce la interfaz de un sistema externo a una interfaz propia, para que la lógica no dependa de él. **P10 Retry + Timeout:** repite una llamada que falló por una causa transitoria y corta la espera al superar un tiempo máximo.	Solo la Integration Layer hace llamadas HTTP salientes (RestClient); cada cliente traduce la respuesta a DTO propios antes de devolverla a la Logic Layer, y lleva @Retry y @TimeLimiter.

2. Atributos de Calidad No Observables

Atributo de Calidad	Descripción	Tácticas / Patrón de Diseño	Dónde se aplica
Adecuación funcional: • Completitud funcional	Grado en el que el conjunto de funcionalidades del producto cubre todas las tareas y los objetivos de usuario especificados.	**P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código; cada HU tiene su operación en él. **P22 Contenerización + pipeline CI/CD:** la API y sus dependencias se empaquetan en una imagen que se construye, prueba y despliega automáticamente.	Las 12 HU tienen endpoint o proceso asignado (tablas 1 y 11), incluidos los dos endpoints mínimos de la sección 11 del caso: POST /patentes/busqueda y GET /patentes/estado. Se evidencia ejecutando una prueba funcional por cada criterio de aceptación, en el pipeline y contra la API desplegada: la funcionalidad está completa cuando todas pasan.
Adecuación funcional: • Corrección funcional	Capacidad del producto o sistema para proveer resultados exactos cuando es usado por los usuarios especificados.	**P06 Validator:** verifica obligatoriedad, formato y rangos (@NotNull, @Size, @Pattern) antes de procesar; si un dato no cumple, la petición falla de inmediato con 400. **P17 State (máquina de estados):** cada entidad tiene estados definidos y solo se permiten las transiciones válidas entre ellos.	Cada HU responde exactamente lo que dicen sus criterios de aceptación, tanto en éxito como en fallo (código HTTP y mensaje). Se evidencia ejecutando las pruebas funcionales de cada criterio contra la API desplegada.

Adecuación funcional: • Pertinencia funcional	Capacidad del producto software para proporcionar un conjunto de funciones que facilitan la consecución de tareas y objetivos de usuario especificados.	**P08 Planificador dinámico:** cada ejecución del barrido se programa con la configuración vigente, sin reiniciar la API. **P17 State (máquina de estados):** cada entidad tiene estados definidos y solo se permiten las transiciones válidas entre ellos; el descarte de un falso positivo es una transición reversible.	Cada función responde a una HU priorizada del PB (tabla 1) y automatiza una tarea: el barrido corre solo con la frecuencia y los criterios del administrador (HU-01, HU-06, HU-10), y Fabricación y Materiales/Fórmulas reciben el estado y el dictamen sin intervención manual (HU-03, HU-04). El administrador solo decide sobre los conflictos: los filtra (HU-07) y descarta o reactiva falsos positivos (HU-09).
Eficiencia de desempeño: • Comportamiento temporal	Grado en que un producto realiza sus funciones de forma que el tiempo de respuesta y el ratio de rendimiento cumple los requisitos especificados.	**P11 Solicitud-respuesta asíncrona (callback):** responde de inmediato que la solicitud fue aceptada, procesa en segundo plano (@Async) y entrega el resultado después en otra petición. **P16 Modelo de lectura precalculado:** el resultado que más se consulta se calcula al escribir y se guarda listo para leer. **P15 Cache-aside:** se consulta primero la caché; si el dato no está, se lee de la base de datos y se guarda en caché. **P10 Retry + Timeout:** repite una llamada que falló por una causa transitoria y corta la espera al superar un tiempo máximo.	HU-04 responde en ≤ 1 s; HU-03 responde 202 dentro de los 5 s que espera Materiales/Fórmulas y entrega el dictamen en ≤ 30 s; HU-02 responde o se corta a los 20 s. Se mide con el temporizador http.server.requests de Micrometer en Grafana.

Eficiencia de desempeño: • Capacidad	Grado en que el producto cumple los requisitos relativos a límites máximos para un parámetro (ítems almacenados, usuarios concurrentes, ancho de banda de comunicaciones...).	**P20 Paginación:** devuelve los resultados por bloques de tamaño fijo en lugar de todos a la vez. **P16 Modelo de lectura precalculado:** el resultado que más se consulta se calcula al escribir y se guarda listo para leer. **P23 Métricas y monitoreo:** la API expone métricas que un recolector consulta periódicamente y un tablero muestra.	Un barrido completo procesa todos las fórmulas activos sin agotar la memoria ni las conexiones de la t3.small (2 vCPU, 2 GiB), y los listados responden igual con 20 o con miles de registros. Se verifica con las métricas de la JVM y de HikariCP en Grafana durante un barrido.
Compatibilidad: • Interoperabilidad	Capacidad de dos o más sistemas o componentes para intercambiar información y utilizar la información intercambiada.	**P09 Adapter / Gateway:** traduce la interfaz de un sistema externo a una interfaz propia, para que la lógica no dependa de él. **P11 Solicitud-respuesta asíncrona (callback):** responde de inmediato que la solicitud fue aceptada, procesa en segundo plano (@Async) y entrega el resultado después en otra petición. **P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código.	Fabricación obtiene el estado del fórmula (HU-04), Materiales/Fórmulas recibe y entiende el dictamen por callback (HU-03) y Notificaciones acepta las alertas (HU-12). Se evidencia en las pruebas de integración con los equipos 1, 3 y 5.

Capacidad de interacción: • Reconocibilidad de la adecuación	Capacidad del producto que permite al usuario entender si el software es adecuado para sus necesidades.	**P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código. **P07 Manejador global de excepciones:** un único @RestControllerAdvice captura las excepciones que suben desde cualquier capa y las convierte en una respuesta de error uniforme.	Un consumidor entiende, sin preguntar, qué hace cada endpoint y qué responde. Ante un 400 por una clasificación IPC inválida o un 404 por un conflictoId inexistente (HU-08), recibe un ProblemDetail (RFC 9457, que reemplaza al RFC 7807) que indica el error exacto sin exponer trazas de código.
Capacidad de interacción: • Aprendizabilidad	Capacidad del producto que permite al usuario aprender su funcionamiento dentro de un tiempo especificado.	**P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código. **P07 Manejador global de excepciones:** un único @RestControllerAdvice captura las excepciones que suben desde cualquier capa y las convierte en una respuesta de error uniforme.	Un integrante de otro equipo aprende a usar la API desde Swagger UI sin documentación aparte: las rutas siguen un mismo esquema (/patentes/{recurso}) y todos los errores tienen la misma forma, así que lo aprendido en un endpoint sirve para los demás.

Capacidad de interacción: • Operabilidad	Capacidad del producto que permite al usuario operarlo y controlarlo con facilidad.	**P08 Planificador dinámico:** cada ejecución del barrido se programa con la configuración vigente, sin reiniciar la API. **P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código.	El administrador cambia la frecuencia o los criterios y el cambio aplica en la siguiente ejecución sin reiniciar la API ni interrumpir la actual (HU-06, HU-10, DA-03); puede explorar y probar todos los endpoints desde Swagger UI.
Capacidad de interacción: • Involucración del usuario	Capacidad del producto de presentar sus funciones e información de forma atractiva y motivadora, fomentando la interacción continua.	**P24 Contrato OpenAPI:** el contrato REST se describe en una especificación estándar generada desde el código. **P12 Eventos de dominio / Observer asíncrono:** quien detecta un hecho publica un evento y los interesados lo procesan aparte (@Async), sin bloquear al emisor. **P23 Métricas y monitoreo:** la API expone métricas que un recolector consulta periódicamente y un tablero muestra.	Sin interfaz gráfica (sección 8 del caso), la interacción ocurre en Swagger UI y en los tableros de Grafana: el administrador ve de forma clara los conflictos detectados y el resultado de cada barrido. Las alertas automáticas (HU-12) lo llevan a revisar un conflicto nuevo cuando aparece, sin tener que consultar la API a cada rato.

Fiabilidad: • Capacidad de recuperación	Capacidad del producto software para recuperar los datos directamente afectados y reestablecer el estado deseado del sistema en caso de interrupción o fallo.	**P28 Health Check:** un endpoint informa si la API y sus dependencias están operativas; quien la ejecuta lo consulta periódicamente y actúa si falla. **P13 Idempotencia:** repetir la misma operación produce el mismo resultado que ejecutarla una sola vez.	Tras una caída, el contenedor vuelve a estar disponible, el siguiente barrido revisa de nuevo las fórmulas NO_PROCESADO y no se repiten alertas ya enviadas (HU-01, HU-12). Se comprueba deteniendo el contenedor o la base de datos en un ensayo y revisando el historial.
Seguridad: • Resistencia	Capacidad de mantener la operación de un producto bajo condiciones de ataque de un actor malicioso.	**P06 Validator:** verifica obligatoriedad, formato y rangos (@NotNull, @Size, @Pattern) antes de procesar; si un dato no cumple, la petición falla de inmediato con 400. **P20 Paginación:** devuelve los resultados por bloques de tamaño fijo en lugar de todos a la vez. **P10 Retry + Timeout:** repite una llamada que falló por una causa transitoria y corta la espera al superar un tiempo máximo.	Ante payloads malformados, peticiones de listados masivos o una dependencia lenta, la API rechaza con 400 lo inválido y sigue atendiendo las demás peticiones en su tiempo normal. Se verifica con peticiones inválidas y de alto volumen desde Postman.

Flexibilidad: • Adaptabilidad	Capacidad del producto que le permite ser adaptado de forma efectiva y eficiente a diferentes entornos determinados de hardware, software, operacionales o de uso.	**P22 Contenerización + pipeline CI/CD:** la API, la JVM y sus dependencias viajan en la misma imagen, que corre igual en cualquier host con Docker Engine (portátil de desarrollo, EC2 u otro proveedor de nube). **P21 Configuración externalizada:** los valores que cambian entre entornos se leen de variables de entorno y perfiles, no del código.	Llevar Patent Sweep a otro entorno (p. ej., de AWS a Google Cloud) no exige cambiar el código: se ejecuta la misma imagen con Docker Compose y solo cambian los valores propios del entorno (URL de PostgreSQL y de las otras API, credenciales, puertos), que llegan por variables de entorno y perfiles de Spring.
Protección: • Restricción operativa	Capacidad de un producto para limitar su funcionamiento a unos parámetros o estados seguros cuando se enfrenta a un peligro operativo.	**P17 State (máquina de estados):** cada entidad tiene estados definidos y solo se permiten las transiciones válidas entre ellos. **P16 Modelo de lectura precalculado:** el resultado que más se consulta se calcula al escribir y se guarda listo para leer.	No se fabrica una fórmula con conflictos vigentes o NO_PROCESADO: Fabricación consulta GET /patentes/estado antes de producir y recibe el estado en ≤ 1 s (HU-04), y Materiales/Fórmulas conserva el dictamen INFRACCION de la evaluación (HU-03). Se evidencia en la prueba de integración con el equipo 3, intentando producir una fórmula que tiene un conflicto DETECTADO.

Protección: • Integración segura	Capacidad de un producto para mantener la seguridad durante y después de la integración con uno o varios componentes	**P09 Adapter / Gateway:** traduce la interfaz de un sistema externo a una interfaz propia, para que la lógica no dependa de él. **P10 Retry + Timeout:** repite una llamada que falló por una causa transitoria y corta la espera al superar un tiempo máximo.	Si Materiales/Fórmulas o Notificaciones no responden, Patent Sweep sigue atendiendo sus endpoints y deja el dictamen o la alerta pendiente, sin quedar en un estado inconsistente. Se comprueba apagando la API externa durante un barrido.

2.2 Patrones de diseño

La tabla 7 lista los patrones arquitectónicos, de diseño y las tácticas que se aplican, el problema que resuelve cada uno y dónde se usa en Patent Sweep.

Tabla 7. Patrones de diseño y tácticas

ID	Patrón / táctica	Tipo	Problema que resuelve	Aplicación en Patent Sweep
P01	Arquitectura multicapa	Arquitectura	Mezcla de responsabilidades.	Controller – Service – Integration – Repository (todas las HU).
P02	SOA / API REST	Arquitectura	Hub monolítico no escalable (Caso 1 y 3).	API independiente con base de datos y contrato HTTP/JSON propios.
P03	Inyección de dependencias (IoC)	Diseño	Acoplamiento a implementaciones concretas; pruebas difíciles.	Inyección por constructor (Spring); mocks en pruebas unitarias.
P04	Repository / DAO	Diseño	Lógica de persistencia dispersa.	Spring Data JPA; Specifications para filtros (HU-05, HU-07).
P05	DTO + Mapper	Diseño	Exponer entidades y acoplar el contrato al modelo.	Request/Response por endpoint (todas).

ID	Patrón / táctica	Tipo	Problema que resuelve	Aplicación en Patent Sweep
P06	Validator	Diseño / táctica	Datos inválidos llegan a la lógica.	Bean Validation + validación de catálogo IPC y rangos (HU-02, 03, 06, 09, 10).
P07	Manejador global de excepciones	Diseño / táctica	Errores heterogéneos y try/catch disperso.	@RestControllerAdvice → ProblemDetail con el mensaje exacto de cada criterio.
P08	Planificador dinámico	Táctica	Cambiar la frecuencia sin reiniciar.	SchedulingConfigurer + Trigger que lee la configuración activa (HU-01, HU-06).
P09	Adapter / Gateway	Diseño (GoF) / integración	Fuente de patentes no definida; dependencia de API externas.	FuentePatentesPort, MaterialesClient, NotificacionesClient (HU-01, 02, 03, 12).
P10	Retry + Timeout	Táctica de disponibilidad	Fallos transitorios y servicios lentos.	Resilience4j: 3 reintentos con 5 s; límite de 20 s (HU-02) y 30 s (HU-03).
P11	Solicitud-respuesta asíncrona (callback)	Integración	Evaluación de hasta 30 s frente a un consumidor que espera 5 s.	202 Accepted + POST /formulas/{id}/dictamen-patente (HU-03).
P12	Eventos de dominio / Observer asíncrono	Diseño	La notificación no debe detener el barrido.	ConflictoDetectadoEvent + @TransactionalEventListener + @Async (HU-12).
P13	Idempotencia	Táctica	Notificaciones o descartes duplicados.	Estado de notificación por conflicto; 409 si ya fue descartado (HU-09, HU-12).
P14	Aislamiento de fallos (bulkhead lógico)	Táctica	Una fórmula fallida detiene todo el barrido.	Fallo capturado por fórmula → NO_PROCESADO y se continúa (HU-01).
P15	Cache-aside	Táctica de desempeño	Consultar el catálogo IPC/CPC en cada petición.	Caffeine + @Cacheable sobre el catálogo estático (HU-02, 03, 10).
P16	Modelo de lectura precalculado	Táctica de desempeño	Responder en 1 s sin búsqueda en tiempo real.	Estado de fórmula actualizado al cerrar cada revisión (HU-04).
P17	State (máquina de estados)	Diseño (GoF)	Transiciones inválidas.	Estados de conflicto, notificación, fórmula y barrido (HU-01, 04, 09, 12).

ID	Patrón / táctica	Tipo	Problema que resuelve	Aplicación en Patent Sweep
P18	Registro de auditoría append-only	Táctica de seguridad	Pérdida de trazabilidad.	Historial de revisiones y de configuraciones (HU-05, 06, 10).
P19	Correlation ID (traceId)	Táctica de observabilidad	Rastrear un error entre API.	Micrometer Tracing → MDC → logs y ProblemDetail.
P20	Paginación	Táctica de desempeño	Respuestas grandes.	50 por página (HU-01); 20 por página (HU-05, HU-07).
P21	Configuración externalizada	Táctica	Cambios de entorno que exigen recompilar.	Perfiles de Spring, variables de entorno y secretos de GitHub.
P22	Contenerización + pipeline CI/CD	DevOps	Despliegues manuales propensos a error.	Dockerfile multi-stage; GitHub Actions + SonarCloud → EC2 (Caso §6).
P23	Métricas y monitoreo	Observabilidad	Falta de visibilidad en producción (Caso 3).	Actuator + Micrometer → Prometheus → Grafana (Caso §7).
P24	Contrato OpenAPI	Integración	Malentendidos entre equipos.	springdoc-openapi + Swagger UI.
P25	Circuit Breaker	Táctica de disponibilidad	Llamadas repetidas a una API caída retienen hilos y retrasan el barrido.	Resilience4j CircuitBreaker en MaterialesClient, NotificacionesClient y el adaptador de la fuente (HU-01, 02, 03, 12).
P26	Autenticador + autorizador (token de sesión)	Táctica de seguridad	Cualquiera puede invocar los endpoints de administración; HU-09 exige una sesión autenticada.	Spring Security: POST /auth/login emite un JWT y un filtro lo valida en cada petición (DA-11).
P27	Control de acceso basado en roles (RBAC)	Táctica de seguridad	Un usuario autenticado podría ejecutar operaciones que no le corresponden.	Roles ADMINISTRADOR y SERVICIO en el token; @PreAuthorize por endpoint (HU-03 a HU-11).
P28	Health Check	Táctica de disponibilidad	Una API caída o bloqueada no se detecta hasta que falla una petición.	/actuator/health (incluye PostgreSQL) + healthcheck de Docker Compose y restart: unless-stopped; el pipeline lo verifica tras desplegar.

2.3 Atributos de calidad vs. patrones

La matriz de la tabla 8 relaciona cada patrón con las características de calidad de ISO/IEC 25010:2023 a las que contribuye. Se usan las nueve características del modelo para que la matriz siga siendo válida con las filas que finalmente queden en la sección 2.1.

Tabla 8. Atributos de calidad vs. patrones

Patrón	AF	ED	CO	CI	FI	SE	PR	MA	FL
P01 Arquitectura multicapa								X	X
P02 SOA / API REST			X					X	X
P03 Inyección de dependencias (IoC)		X						X	
P04 Repository / DAO								X	X
P05 DTO + Mapper			X			X		X	
P06 Validator	X	X		X		X	X		
P07 Manejador global de excepciones				X	X			X	
P08 Planificador dinámico	X			X					
P09 Adapter / Gateway			X				X	X	X
P10 Retry + Timeout		X			X	X	X		
P11 Solicitud-respuesta asíncrona (callback)		X	X		X				
P12 Eventos de dominio / Observer asíncrono		X		X	X		X	X	
P13 Idempotencia	X			X	X	X			
P14 Aislamiento de fallos (bulkhead lógico)					X		X		
P15 Cache-aside		X							
P16 Modelo de lectura precalculado		X			X		X		
P17 State (máquina de estados)	X			X		X	X		
P18 Registro de auditoría append-only						X		X	
P19 Correlation ID (traceId)					X	X		X	
P20 Paginación		X		X		X			

Patrón	AF	ED	CO	CI	FI	SE	PR	MA	FL
P21 Configuración externalizada						X			X
P22 Contenerización + pipeline CI/CD	X		X		X			X	X
P23 Métricas y monitoreo		X		X	X			X	
P24 Contrato OpenAPI	X		X	X				X	X
P25 Circuit Breaker					X				
P26 Autenticador + autorizador (token de sesión)						X			
P27 Control de acceso basado en roles (RBAC)						X			
P28 Health Check					X				
Total	6	9	6	9	12	11	7	13	7

AF: Adecuación funcional · ED: Eficiencia de desempeño · CO: Compatibilidad · CI: Capacidad de interacción · FI: Fiabilidad · SE: Seguridad · PR: Protección · MA: Mantenibilidad · FL: Flexibilidad.

3. Vista Física

La vista física muestra la distribución de la aplicación en componentes de software y sus interacciones, con la misma notación del modelo de referencia del curso: paquetes por capa, dependencias «inject» entre capas, dependencias «use» hacia la capa transversal y las API con las que se integra (figura 2).

Las responsabilidades de cada componente se describen en la tabla 9.

Tabla 9. Componentes de la API Patent Sweep

Componente	Capa	Responsabilidad	HU
BusquedaController / BusquedaService	Presentación / Lógica	Búsqueda manual de patentes para una fórmula (≤ 20 s).	HU-02
EvaluacionController / EvaluacionService	Presentación / Lógica	Recibir la solicitud de Materiales/Fórmulas, responder 202 y evaluar en segundo plano.	HU-03
EstadoFormulaController / EstadoFormulaService	Presentación / Lógica	Exponer el estado precalculado de la fórmula a Fabricación (≤ 1 s).	HU-04

Componente	Capa	Responsabilidad	HU
HistorialController	Presentación	Historial paginado (20) de revisiones por fórmula.	HU-05
ConflictoController / ConflictoService	Presentación / Lógica	Listar, detallar, descartar y reactivar conflictos; recalcular el estado de la fórmula.	HU-07, 08, 09
ConfiguracionController / ConfiguracionService	Presentación / Lógica	Frecuencia y criterios del barrido con historial y una sola configuración activa.	HU-06, 10, 11
BarridoScheduler / BarridoService	Lógica	Programar y ejecutar el barrido; registrar resultado y conteos.	HU-01
MotorCoincidencias	Lógica	Comparar clasificaciones y componentes de la fórmula con las patentes de la fuente.	HU-01, 02, 03
NotificacionService	Lógica	Enviar una notificación por conflicto nuevo con 3 intentos, sin bloquear el barrido.	HU-12
MaterialesClient	Integración	Listado de fórmulas activos y callback del dictamen.	HU-01, 03
NotificacionesClient	Integración	POST /notificaciones hacia el Equipo 5.	HU-12
FuentePatentesPort + adaptador	Integración	Abstraer la fuente de patentes (dataset local o fuente real).	HU-01, 02, 03
IRepository / Model	Acceso a datos	Persistencia JPA de fórmulas, conflictos(con el estado de su notificación), patentes, historial, barridos y configuraciones.	Todas
Utility Layer	Transversal	Validación, errores, seguridad (autenticación y roles), resiliencia, caché, OpenAPI, health check y métricas.	Todas

Por su parte, para ofrecer todas las capacidades que el diseño de la API del departamento de patentes posee, se comenzó planteando el comportamiento y manejo de la información que se va a tratar dentro de la plataforma. Por tal motivo, se estableció un modelo relacional de la base de datos de la API Patent Sweep, que está estructurado para soportar el almacenamiento transaccional, la trazabilidad estricta y los registros de auditoría append-only exigidos por las reglas del negocio. El modelo garantiza la correcta separación de responsabilidades y la integridad referencial en todo el ciclo de vida de la vigilancia tecnológica.

En cuanto a la gestión de fórmulas, la entidad Formula conserva la referencia a cada fórmula vigilada: su código, nombre y descripción, que se copian de Materiales/Fórmulas (la API dueña de esos datos), junto con la información propia de Patent Sweep: el estado de vigilancia, el último dictamen y la fecha de la última revisión. De forma complementaria, la entidad de

evaluación gestiona las solicitudes asíncronas provenientes de Materiales/Fórmulas, registrando el ticket, los componentes evaluados, el dictamen (SIN_INFRACCION o INFRACCION) y el estado del procesamiento (EN_PROCESO, COMPLETADA, FALLIDA o PENDIENTE_DE_ENVIO).

Para la ejecución y trazabilidad, el modelo implementa registros históricos estructurados como tablas append-only con retención de largo plazo. Por un lado, se registran las ejecuciones de los barridos automáticos periódicos junto con sus métricas de control y estados de ciclo de vida. Por otro lado, se archiva el historial detallado de todas las revisiones ejecutadas, diferenciando su origen y asociando el trazo de auditoría, el identificador de correlación y los parámetros evaluados.

En el ámbito de la propiedad intelectual y la gestión de conflictos, el sistema almacena el detalle completo de las patentes candidatas detectadas en la fuente externa, incluyendo títulos, resúmenes, reivindicaciones, fechas clave, jurisdicción y estado de vigencia. A partir de ello, la entidad de conflictos relaciona cada fórmula con su patente detectada bajo una restricción de unicidad estricta, permitiendo gestionar su ciclo de vida entre los estados de detección y descarte, e integrando el motivo y el usuario responsable en caso de falsos positivos.

Finalmente, la configuración del sistema y la seguridad se estructuran mediante un control riguroso de parámetros y acceso. La entidad de configuración guarda la frecuencia del barrido y los criterios de búsqueda (rango de fechas y lista de clasificaciones IPC/CPC), y solo una fila está activa a la vez, conservando inmutable el historial previo; los códigos se validan contra el catálogo de clasificaciones. Así mismo, el módulo de usuarios administra las credenciales y roles del personal autorizado, mientras que el registro de notificaciones, con una sola fila por conflicto, garantiza la idempotencia y el control de reintentos (estados PENDIENTE_NOTIFICACION y NOTIFICADO) en el despacho de alertas hacia el servicio externo correspondiente. En la figura 2 se presenta el diagrama relacional de la base de datos.

Archivo: MER_Sweep_Patent

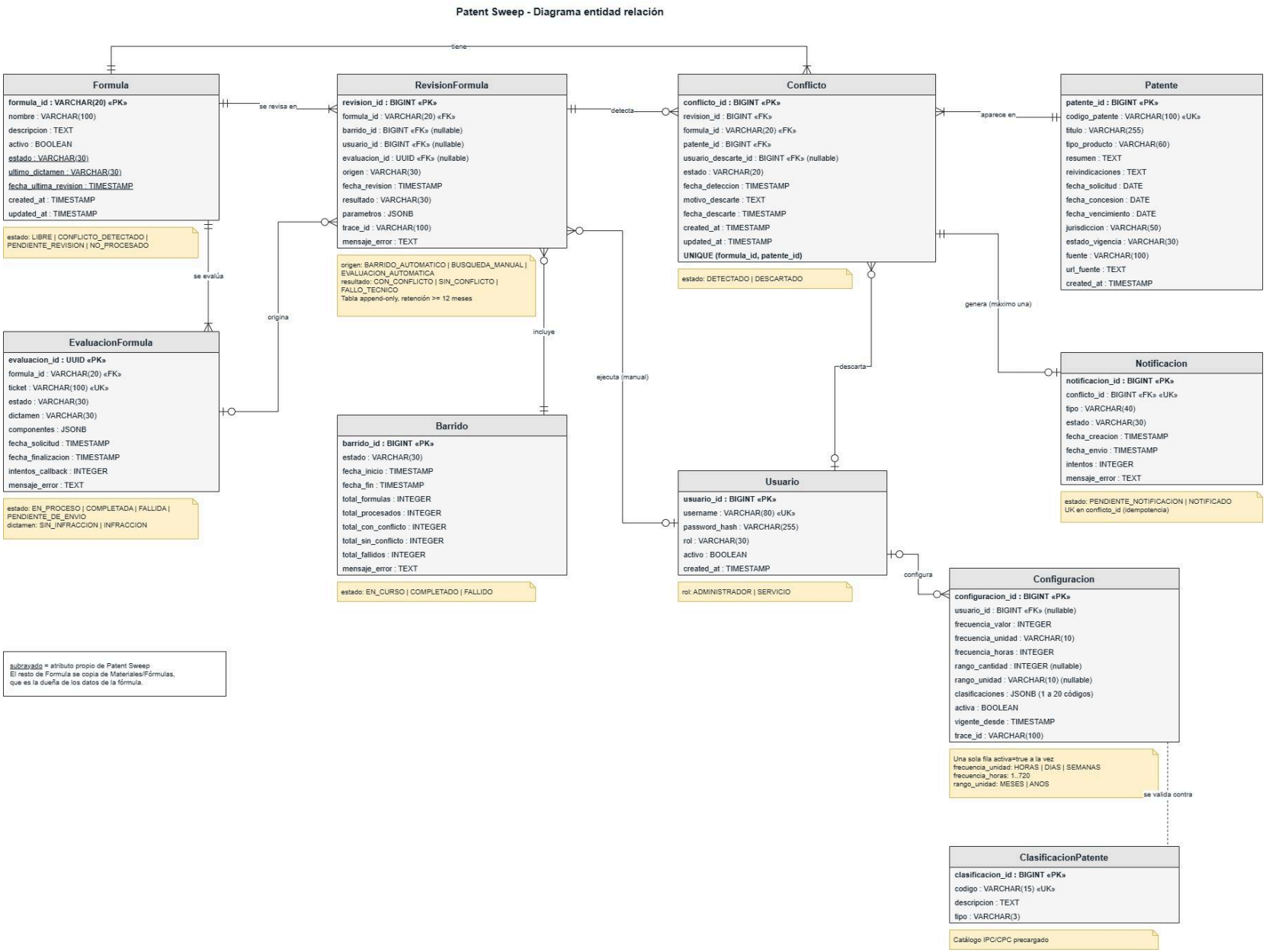


Figura 1. Diagrama entidad relación del componente Model

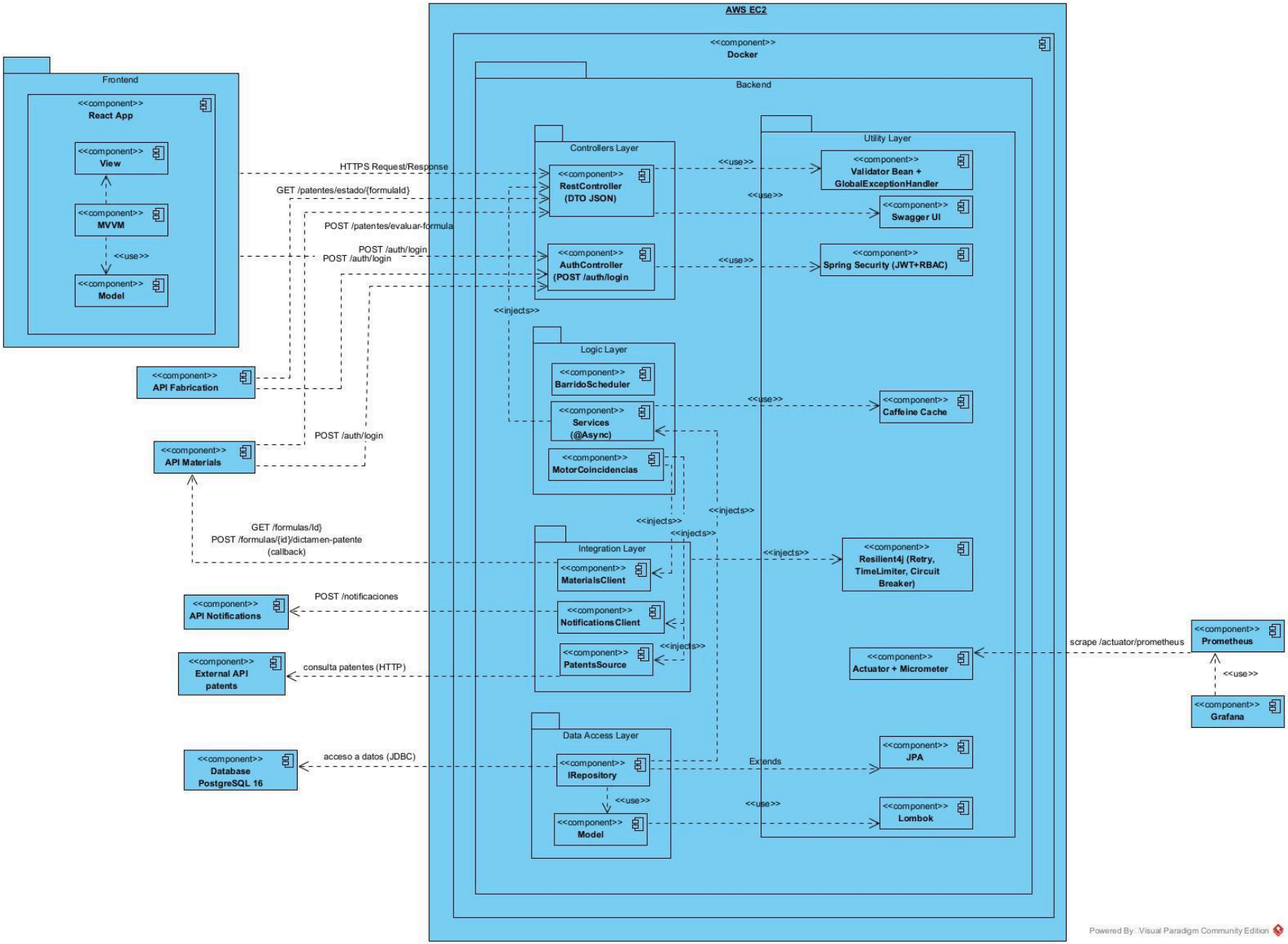


Figura 2. Vista física: Diagrama de componentes de la API Patent Sweep.

4. Vista Funcional o Lógica

4.1 Modos de comunicación

- **Solicitud-respuesta síncrona HTTP/JSON:** consultas y comandos del administrador y de Fabricación (HU-02, 04 a 11).
- **Solicitud-respuesta asíncrona con callback:** evaluación de fórmulas solicitada por Materiales/Fórmulas (HU-03).
- **Eventos internos asíncronos:** ConflictoDetectadoEvent desacopla el envío de notificaciones del barrido (HU-12).
- **Activación temporal:** el planificador dispara el barrido según la frecuencia vigente (HU-01, HU-06).
- **JDBC** hacia PostgreSQL y **pull de métricas** de Prometheus sobre /actuator/prometheus.

4.2 Redes de procesos

Tabla 10. Redes de procesos

Red de procesos	Procesos involucrados	Disparador	Comunicación	Ciclo de vida
Barrido automático	HU-01 barrido; HU-12 alertas; usa la configuración de HU-06 y HU-10	Planificador interno	Interna + HTTP saliente a Materiales/Fórmulas, fuente de patentes y Notificaciones	Barrido EN_CURSO → COMPLETADO o FALLIDO; fórmula NO_PROCESADO ante fallo puntual
Evaluación de fórmulas	HU-03	Materiales/Fórmulas	202 Accepted + callback	Evaluación registrada → dictamen entregado o pendiente
Búsqueda manual	HU-02	Administrador	Síncrona (≤ 20 s)	Petición-respuesta; deja EventoRevision
Consultas	HU-04, 05, 07, 08, 11	Administrador / Fabricación	Síncrona de solo lectura	Sin estado
Gestión de conflictos	HU-09	Administrador	Síncrona	Conflicto DETECTADO ↔ DESCARTADO
Configuración	HU-06, HU-10	Administrador	Síncrona	Nueva configuración activa; la anterior pasa al historial

4.3 Interacciones

La figura 3 muestra el barrido periódico con el aislamiento de fallos por fórmula y el envío de alertas en paralelo; la figura 4, la evaluación asíncrona solicitada por Materiales/Fórmulas.

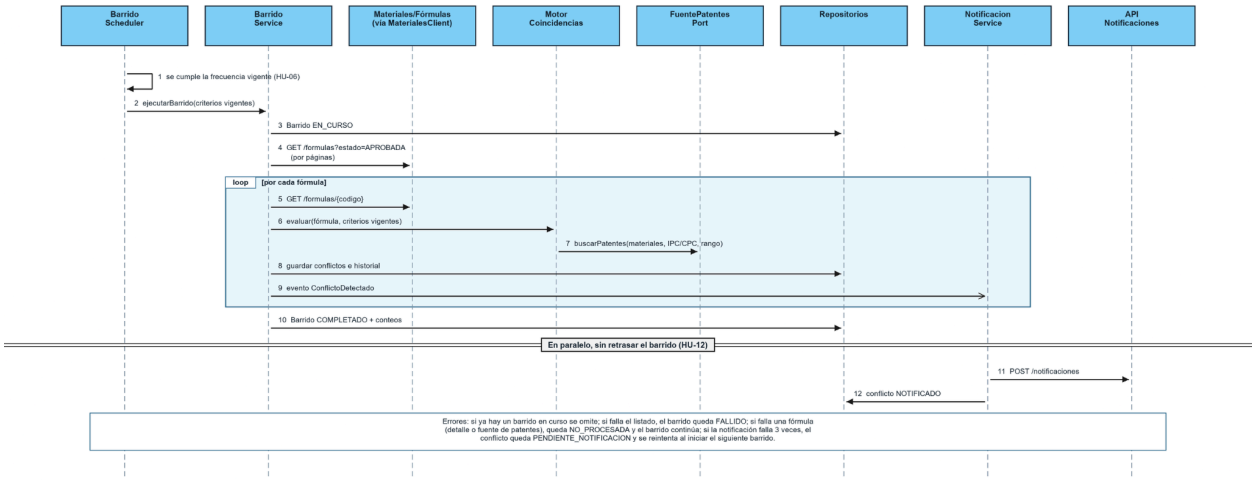


Figura 3. Diagrama de secuencia del barrido periódico y las alertas (HU-01, HU-12).

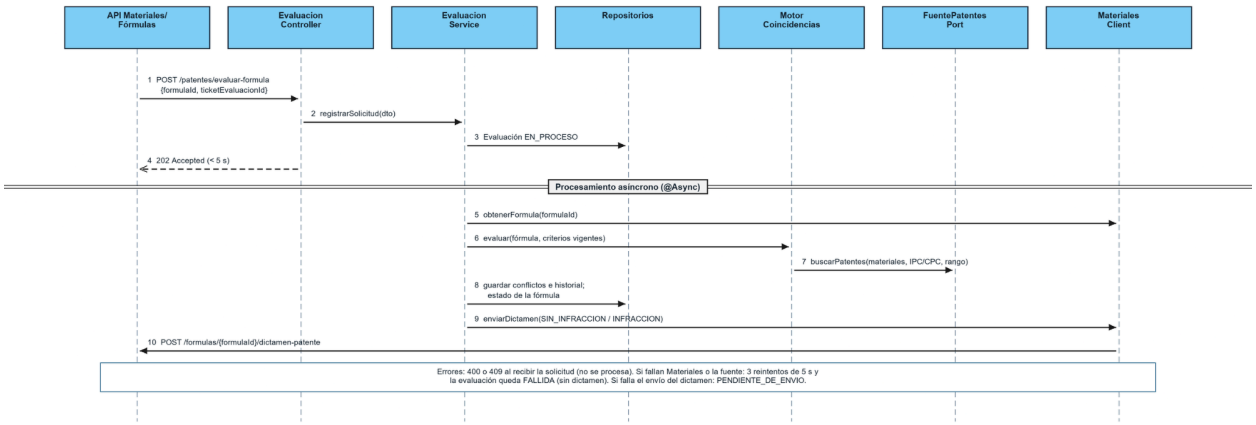


Figura 4. Diagrama de secuencia de la evaluación de una fórmula (HU-03).

4.4 Contrato REST

Tabla 11. Endpoints expuestos por Patent Sweep

Método	Ruta	HU	Consumidor	Respuestas	Origen
POST	/patentes/busqueda	HU-02	Administrador	200, 400, 500	Caso S11 / PB
POST	/patentes/evaluar-formula	HU-03	Materiales/Fórmulas	202*, 400, 500	PB (ruta sin «ñ»; *ver S-03)

Método	Ruta	HU	Consumidor	Respuestas	Origen
GET	/patentes/estado?formulaId={id}	HU-04	Fabricación	200, 400, 404, 500	Caso S11 / PB
GET	/patentes/historial/{formulaId}?page=	HU-05	Administrador	200, 404, 500	PB
PUT	/patentes/configuracion/frecuencia	HU-06	Administrador	200, 400, 500	PB
GET	/patentes/conflictos?estado=&formulaId=&page=	HU-07	Administrador	200, 400, 500	PB
GET	/patentes/conflictos/{conflictoId}	HU-08	Administrador	200, 404, 500	PB
PATCH	/patentes/conflictos/{conflictoId}/descarte	HU-09	Administrador	200, 400, 404, 409, 500	PB
PATCH	/patentes/conflictos/{conflictoId}/reactivacion	HU-09 (crit. 4)	Administrador	200, 404, 409, 500	Propuesto (el PB pide reversibilidad sin ruta)
PUT	/patentes/configuracion/criterios-busqueda	HU-10	Administrador	200, 400, 500	PB
GET	/patentes/configuracion/criterios-busqueda	HU-11	Administrador	200, 500	PB
GET	/patentes/barridos?page=	HU-01 (crit. 5)	Administrador	200, 500	Propuesto (resultado paginado de 50, sin ruta en el PB)
POST	/auth/login	HU-05, HU-09	Administrador, API consumidoras	200, 400, 401, 500	Propuesto (DA-11)
GET	/actuator/health, /actuator/prometheus	—	Pipeline, Prometheus	200	Técnico

Tabla 12. Endpoints consumidos por Patent Sweep

Método	Ruta (API destino)	HU	Estado del contrato
GET	/formulas?estado=APROBADA (Materiales/Fórmulas)	HU-01	Supuesto S-01

Método	Ruta (API destino)	HU	Estado del contrato
POST	/formulas/{id}/dictamen-patente (Materiales/Fórmulas)	HU-03	Definido en el PB de Materiales/Fórmulas (HU-10)
POST	/notificaciones (Notificaciones)	HU-12	Pendiente con el Equipo 5 (S-05)
—	Fuente de patentes (externa)	HU-01, 02, 03	Por definir (S-04)

5. Vista de Despliegue

5.1 Configuración de producción en AWS

La API se despliega como contenedor Docker en una instancia EC2 junto con su base de datos y el stack de monitoreo, orquestados con Docker Compose. El pipeline de GitHub Actions construye, prueba, analiza y publica la imagen, y la despliega por SSH solo si el quality gate de SonarCloud es aprobado (figura 5).

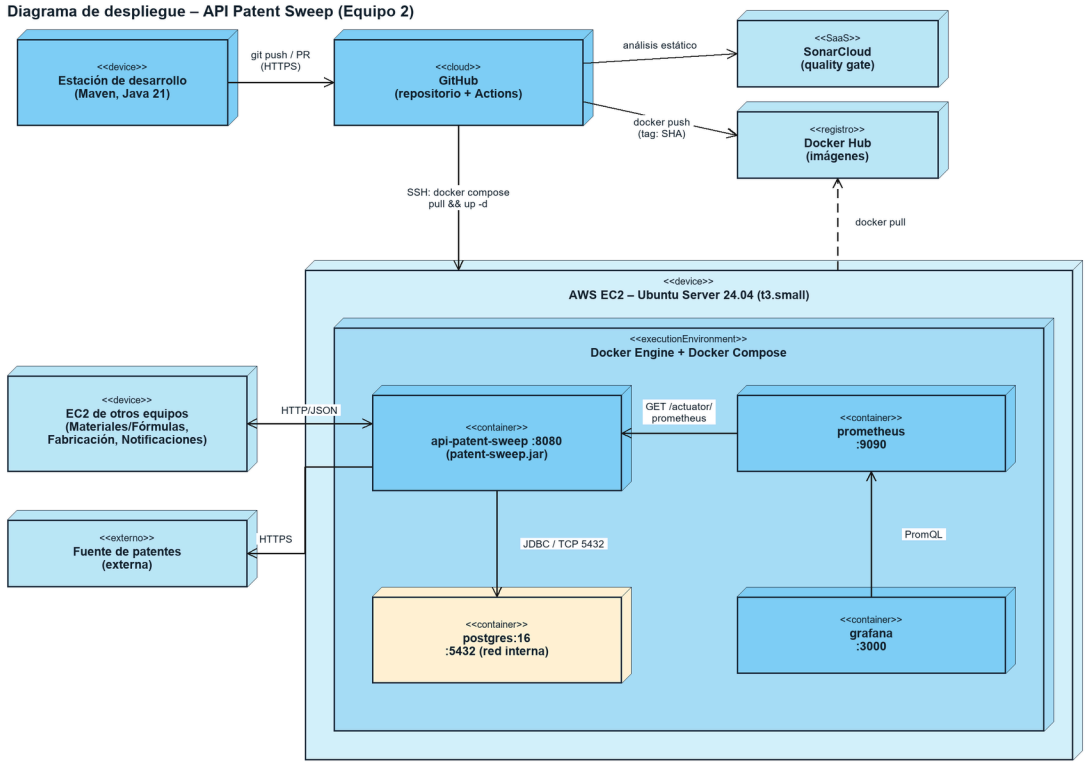


Figura 5. Diagrama de despliegue de la API Patent Sweep.

Tabla 13. Nodos de despliegue

Nodo	Tipo	Software / procesos	Puertos y protocolos
Estación de desarrollo	Dispositivo	IDE, JDK 21, Maven, Git, Docker	HTTPS hacia GitHub
GitHub (repositorio + Actions)	Servicio en la nube	Rama main protegida; workflow ci-cd.yml	HTTPS; SSH (22) hacia EC2
SonarCloud	SaaS	Análisis estático y quality gate	HTTPS
Docker Hub	Registro	Imagen patent-sweep etiquetada con el SHA del commit	HTTPS
EC2 Ubuntu Server 24.04 (t3.small: 2 vCPU, 2 GiB)	Dispositivo	Docker Engine + Compose: api-patent-sweep, postgres, prometheus, grafana, node-exporter	Security Group: 22 (despliegue), 8080 (API), 3000 (Grafana); 5432, 9090 y 9100 solo en la red interna de Docker
EC2 de otros equipos	Dispositivo	API Materiales/Fórmulas, Fabricación y Notificaciones	HTTP/JSON
Fuente de patentes	Externa	Por definir (S-04)	HTTPS

5.2 Mapa de procesos por nodo

Tabla 14. Mapa de procesos por nodo

Proceso (vista lógica)	Nodo / contenedor	Observación
Atención de peticiones REST	api-patent-sweep	Hilos del servidor web embebido (Tomcat).
Barrido periódico	api-patent-sweep	Hilo del planificador; si se escala a varias réplicas, se requiere un bloqueo distribuido (p. ej., ShedLock) para que solo una ejecute el barrido.
Evaluaciones y notificaciones asíncronas	api-patent-sweep	ThreadPoolTaskExecutor dedicado para no competir con las peticiones REST.
Persistencia	postgres	Volumen Docker persistente; respaldo con pg_dump.
Recolección de métricas	prometheus, node-exporter	Scrape cada 15 s de la API y del sistema operativo.
Tableros	grafana	Tableros de la JVM, del servidor y de negocio.

Proceso (vista lógica)	Nodo / contenedor	Observación
Build, pruebas y análisis	GitHub Actions runner / SonarCloud	Fuera de AWS.